# HLD Level 1.9

**Concurrency, Worker Pools, Bounded Queues & Backpressure — C++ / Systems Interview Notes**

> **Core idea: concurrency is not just 'use threads.' Design how work flows, where it waits, how much can be in flight, and what happens when demand exceeds capacity.**

## 1. Concurrency vs parallelism

**Concurrency** means tasks make progress during overlapping periods. **Parallelism** means tasks actually execute simultaneously on multiple cores/hardware units.

## 2. Processes vs threads

Processes provide stronger isolation and separate address spaces. Threads share process memory and communicate cheaply, but shared mutable state requires synchronization.

## 3. Worker pools

```
Requests -> [bounded work queue] -> Worker 1
-> Worker 2
-> Worker N
```

A bounded pool reuses threads instead of creating an unbounded thread per request. CPU-bound pool sizing is constrained by compute resources; I/O-bound workloads may support more concurrency because tasks spend time waiting. Measure rather than use a universal number.

## 4. Producer-consumer

```
Producer -> Queue -> Consumer(s)
```

A queue decouples producer and consumer timing, but queued work still consumes memory and adds latency.

## 5. Queueing latency

```
total latency = queue wait + processing + downstream wait
```

Even if processing itself is fast, a deep queue can make end-to-end latency unacceptable.

## 6. Throughput mismatch

```
producer = 50,000/s
consumer = 40,000/s
backlog growth = 10,000/s
```

> **A larger queue does not fix a permanent throughput mismatch. It only postpones overload while increasing latency.**

## 7. Bounded queues

A bounded queue gives memory and latency an explicit limit. When full, the architecture must choose a policy.

## 8. Full-queue policies

Options include blocking/throttling the producer, rejecting new work, dropping newest, dropping oldest, coalescing replaceable work, prioritizing, spilling to durable storage, or scaling consumers. The requirement determines the answer.

## 9. Freshness: drop-oldest

```
Capture -> [bounded frame queue] -> Inference
full? discard oldest queued frame
```

For live detection, newer frames can be more valuable than stale frames. This matches your camera pipeline. For archival video, financial orders, or durable jobs, dropping may be unacceptable.

## 10. Backpressure

Backpressure prevents downstream overload from turning into unlimited upstream backlog. It can be blocking, flow control, throttling, rejection, reduced production rate, or quality degradation.

```
Fast producer -> backpressure/control -> bounded downstream
```

## 11. Buffering vs backpressure

Buffering absorbs short bursts. Backpressure handles sustained mismatch. If average arrival rate stays above service rate, an ever-larger buffer only delays failure.

## 12. Little's Law intuition

```
L ~= lambda * W
1000 req/s x 0.2 s ~= 200 requests in system
```

At the same throughput, higher latency means more in-flight work and therefore more resource consumption.

## 13. Mutex

A mutex protects shared state. Keep critical sections small and avoid holding a lock across slow I/O where possible.

## 14. Condition variable

```
queue empty -> consumer sleeps
producer pushes -> notify -> consumer wakes
```

Condition variables avoid burning CPU in a polling loop. This is the correct mental model for your producer-consumer implementation.

## 15. Atomics

Atomics are useful for suitable indivisible shared operations, but they do not automatically protect complex invariants spanning multiple variables.

## 16. Lock contention

```
Worker 1 --\
Worker 2 ----> ONE global mutex -> serialized section
Worker N --/
```

If many workers wait for the same lock, adding more threads may reduce performance due to contention and scheduling overhead.

## 17. Sharding shared state

Partitioning a hot structure into independent shards/locks can reduce contention when most operations touch only one shard.

```
hash(key) -> shard0 [lock + map]
-> shard1 [lock + map]
-> shard2 [lock + map]
```

## 18. Deadlock

```
T1 holds A -> waits B
T2 holds B -> waits A
```

Avoid inconsistent lock ordering, unnecessary nested locks, and holding locks while invoking unknown/blocking code.

## 19. Starvation & fairness

A task may wait indefinitely while other tasks continue. Scheduling, priority and queue/lock policy affect fairness.

## 20. Busy waiting

```
BAD: while(queue.empty()) {}
BETTER: wait on condition/event
```

Busy waiting consumes CPU. It is only appropriate in specialized cases where the latency/resource trade-off is deliberately justified.

## 21. Blocking vs event-driven I/O

Blocking I/O is simple but ties execution context to waiting. For very many mostly idle connections, non-blocking/event-driven I/O is often more scalable.

```
many sockets -> epoll/event loop -> ready work -> worker pool
```

## 22. Event loop + worker pool

A common C++ server design lets an event loop handle socket readiness while CPU-heavy or blocking work is dispatched through a bounded queue to workers.

## 23. CPU-bound work

If inference/compression saturates CPU or accelerator capacity, adding threads beyond the real compute resource may only add contention.

## 24. I/O-bound work

Overlapping network/disk waits can improve utilization, but concurrency still needs bounds so downstream systems, FDs and memory are not overwhelmed.

## 25. Batching

Batching amortizes lock, syscall, network, serialization, transaction or accelerator overhead. The trade-off is additional wait latency and more complex partial failure handling.

## 26. Cancellation

If a request deadline expires or client disconnects, continuing expensive work may waste capacity. Propagate cancellation when safe and supported.

## 27. Graceful shutdown

```
stop intake -> mark stopping -> notify workers -> drain/abandon by policy -> join -> release
resources
```

No worker should remain blocked forever after shutdown begins.

## 28. C++ ownership

```
producer creates unique_ptr
move into queue
worker receives ownership
```

RAII, unique_ptr and move semantics make queued-object lifetime explicit. Use shared_ptr only when ownership is genuinely shared; it adds reference-count overhead and can obscure lifetime.

## 29. Large buffers and copying

Large video/frame buffers are expensive to copy. Move ownership, reuse buffer pools, share immutable buffers, or use zero-copy approaches when profiling shows the need.

## 30. Queue memory

```
70 frames x 3 MB ~= 210 MB payload alone
```

Actual memory is higher after object metadata, allocator overhead and additional pipeline buffers. Queue capacity is therefore a memory-budget decision.

## 31. Metrics

Track queue depth, queue wait time, throughput, drop/rejection count, worker utilization, lock contention, CPU/memory and P50/P95/P99 latency.

## 32. Tail latency

Averages can hide scheduling, lock and I/O spikes. P95/P99 are especially useful for identifying unstable pipelines.

## 33. Your camera pipeline

```
Capture -> Preprocess -> Inference -> Postprocess
bounded queues between stages
```

At 45 FPS, if downstream stages cannot sustainably keep up, queue depth and latency rise and resource pressure can destabilize the system. A sustainable 30 FPS can produce better, more predictable real-time behavior.

## 34. Interview decision framework

**1. What work can run concurrently?**
**2. CPU-bound or I/O-bound?**
**3. Worker pool, event loop, or combination?**
**4. Where are queues?**
**5. Are they bounded?**
**6. What happens when full?**
**7. Completeness or freshness?**
**8. What shared state exists?**
**9. What synchronization protects it?**
**10. Where can contention occur?**
**11. Who owns buffers/tasks?**
**12. Can copies be reduced?**
**13. Cancellation?**
**14. Shutdown?**
**15. Which metrics expose saturation?**

## 35. Common mistakes

**More threads always means more throughput:** false.
**Unbounded queue:** overload becomes memory/latency growth.
**Huge queue fixes mismatch:** false.
**Atomics replace locks:** false for complex invariants.
**Busy wait for ordinary work:** wastes CPU.
**Drop data everywhere:** wrong for durable workloads.
**Hold mutex during slow I/O:** can serialize workers.
**shared_ptr everywhere:** unclear ownership and overhead.
**Average latency is enough:** tail latency matters.

## 36. Practice scenarios

**1.** Producer creates 30k tasks/s; workers sustain 20k/s. What happens and what choices do you have?

**2.** Camera receives 45 FPS but inference sustains 30 FPS. Would you use an unbounded queue?

**3.** C++ server has 100k mostly idle sockets. Would you create 100k threads?

**4.** Eight workers mostly wait on one global mutex. Will adding eight more workers necessarily help?

**5.** When should a producer block versus drop/reject?

**6.** Why use condition_variable rather than repeatedly polling an empty queue?

**7.** Queue holds 70 RGB frames of ~3 MB each. What concern appears?

**8.** How do you gracefully stop workers without leaving them asleep forever?

## 37. 60-second revision

**Worker pool: bounded reusable workers.**
**Bounded queue: explicit memory/latency limit.**
**Backpressure: constrain upstream.**
**Drop-oldest: useful when freshness beats completeness.**
**Mutex: protect shared critical state.**
**Condition variable: sleep/wake instead of polling.**

**Contention: shared resource can serialize workers.**
**Event-driven I/O: useful for many mostly idle connections.**
**Batching: throughput vs latency.**
**RAII/ownership: explicit lifetime/cleanup.**
**P95/P99: reveal tail latency.**

## 38. Ready for Level 1.10 when...

**You can design a bounded producer-consumer pipeline, choose an overload policy from requirements, reason about worker pools/event loops, identify contention and resource limits, and explain C++ ownership and shutdown behavior.**